Entwicklerdokumentation: NFA Visualisierung

Projekt: Visualizing Lexical Analysis - Subset Construction
Tech Stack: React + TypeScript + Vite
Zielgruppe: Entwickler, Maintainer

---

1. Setup & Installation

1.1 Voraussetzungen

Software:

- Node.js 18+ (LTS empfohlen)
- npm 9+ oder yarn
- Git
- VS Code (empfohlen)

VS Code Extensions (empfohlen):

- ESLint
- Prettier
- TypeScript and JavaScript Language Features
- Tailwind CSS IntelliSense

1.2 Repository klonen

```
git clone https://gitlab.uni-
marburg.de/datengta/visualizing_lexical_analysis_subsetconstruction.git
cd visualizing_lexical_analysis_subsetconstruction
```

1.3 Dependencies installieren

```
npm install
```

Wichtige Dependencies:

```
{
  "react": "^19.1.1",
  "typescript": "~5.9.3",
  "vite": "^7.2.2",
  "reactflow": "^11.11.4",
  "dagre": "^0.8.5",
  "vitest": "^2.1.8"
}
```

1.4 Development Server starten

```
npm run dev
```

→ Öffnet `http://localhost:5173`

---

2. Projektstruktur

```
visualizing_lexical_analysis_subsetconstruction/
├── public/                      # Statische Assets
│   └── examples/               # Beispiel-AEF-Dateien
│       ├── example_nfa.aef
│       └── example_combined_state.aef
│
├── src/                         # Quellcode
│   ├── components/            # React-Komponenten
│   │   ├── Controls/
│   │   │   └── StepControls.tsx        # Algorithmus-Steuerung
│   │   ├── FileInput/
│   │   │   ├── DSLInput.tsx            # Eingabe-Komponente
│   │   │   └── DSLInput.test.tsx       # Component Tests
│   │   └── GraphViewer/
│   │       ├── GraphViewer.tsx         # Graph-Visualisierung
│   │       ├── GraphViewer.test.tsx    # Component Tests
│   │       └── graphStyles.css         # Spezifische Styles
│   │
│   ├── core/                  # Business Logic
│   │   ├── algorithm/
│   │   │   └── subsetConstruction.ts   # Hauptalgorithmus
│   │   ├── models/
│   │   │   └── types.ts               # TypeScript-Typen
│   │   └── parser/
│   │       ├── dslParser.ts           # AEF-Parser
│   │       └── dslParser.test.ts      # Unit Tests
│   │
│   ├── hooks/                 # Custom React Hooks
│   ├── pages/                 # Seiten-Komponenten
│   │   └── Home.tsx
│   ├── utils/                 # Hilfsfunktionen
│   │   └── helpers.ts
│   │
│   ├── App.tsx               # Haupt-App-Komponente
│   ├── main.tsx              # Entry Point
│   ├── index.css             # Globale Styles
│   └── test-setup.ts         # Test-Konfiguration
│
├── tests/                     # Zusätzliche Test-Suites
│   ├── dslParser.test.ts
│   ├── strict-validation.test.ts
│   ├── consistency-validation.test.ts
│   ├── combined-state-notation.test.ts
│   ├── spaces-and-duplicates.test.ts
│   └── example-nfa.test.ts
│
├── documentation/             # Projekt-Dokumentation
│   ├── Projektdokumentation.md
│   ├── Benutzerdokumentation.md
│   ├── Entwicklerdokumentation.md
│   ├── COMBINED_STATE_NOTATION.md
```

```
│   ├── SPACES_AND_DUPLICATES.md
│   └── VALIDATION_RULES.md
│
├── .gitlab-ci.yml            # CI/CD Pipeline
├── package.json              # Projekt-Konfiguration
├── tsconfig.json             # TypeScript-Konfiguration
├── vite.config.ts            # Vite-Konfiguration
├── vitest.config.ts          # Test-Konfiguration
└── tailwind.config.js        # TailwindCSS-Konfiguration
```

---

3. Architektur

3.1 Komponenten-Hierarchie

```
App.tsx
└── Home.tsx
    ├── DSLInput
    │   ├── Textarea (Eingabe)
    │   ├── FileUpload (Datei laden)
    │   └── ExampleDropdown (Beispiele)
    │
    ├── GraphViewer
    │   ├── ReactFlow (Graph-Rendering)
    │   │   ├── Nodes (Zustände)
    │   │   └── Edges (Transitionen)
    │   ├── MiniMap (Übersicht)
    │   ├── Controls (Zoom/Pan)
    │   └── Background (Raster)
    │
    └── StepControls (zukünftig)
        ├── Play/Pause
        ├── Step Forward/Back
        └── Speed Control
```

3.2 Datenfluss

```
┌──────────────┐
│ DSLInput     │
│ (User Input) │
└──────────────┘
       │ Text
       ↓
┌──────────────┐
│ dslParser    │
│ (Parsing)    │
└──────────────┘
       │ NFA
       ↓
┌──────────────┐
│ GraphViewer  │
│ (Display)    │
└──────────────┘

Zukünftig:
       │
       ↓
┌──────────────┐
```

```
┌──────────────┐
│ subset       │
│Construction  │
└──────────────┘
        │ DFA
        ↓
┌──────────────┐
│ GraphViewer  │
│ (Compare)    │
└──────────────┘
```

---

4. Kern-Module

4.1 dslParser.ts

Verantwortlichkeit: AEF-Format → NFA-Objekt

Hauptfunktionen:

```
// Haupt-Entry-Point
function parseDSL(input: string): ParseResult

// Interne Funktionen
function parseAEFFormat(lines: string[]): NFA
function parseTransitionLine(line: string, lineNum: number): ParsedLine
function parseStateToken(token: string): StateInfo
function validateLineFormat(line: string, lineNum: number): void
function finalizeNFA(states, transitions, ...): NFA
```

Validierungsregeln:

1. Zeilen müssen mit `;` enden
2. Genau ein Startzustand (`.`)
3. Mindestens ein Akzeptierender Zustand `()`
4. Korrekte Pfeil-Syntax `-symbol>`
5. Konsistente Zustandsnotationen
6. Keine Leerzeichen in Zustandsnamen
7. Valide Epsilon-Symbole (nur `ε`)

Beispiel-Nutzung:

```
import { parseDSL } from './core/parser/dslParser';

const input = `.q0 -a> (q1);`;
const result = parseDSL(input);

if (result.success) {
  console.log(result.nfa);
} else {
  console.error(result.error);
}
```

4.2 GraphViewer.tsx

Verantwortlichkeit: NFA/DFA Visualisierung

Props:

```
interface GraphViewerProps {
  nfa?: NFA;
  dfa?: DFA;
  interactive?: boolean;
}
```

Interne Funktionen:

```
// Layout-Berechnung
function layoutWithDagre(
  states: string[],
  transitions: Transition[]
): Map<string, Position>

// Node-Erstellung
function createNodes(nfa: NFA, positions: Map): Node[]

// Edge-Erstellung mit Styling
function createEdges(transitions: Transition[]): Edge[]
```

Beispiel-Nutzung:

```
<GraphViewer
  nfa={myNFA}
  interactive={true}
/>
```

4.3 DSLInput.tsx

Verantwortlichkeit: Eingabe und Laden von NFA-Definitionen

Props:

```
interface Props {
  onLoad: (text: string) => void;
  onParseResult: (result: ParseResult) => void;
}
```

State:

```
const [text, setText] = useState("");
const [isLoading, setIsLoading] = useState(false);
const [selectedExample, setSelectedExample] = useState("");
```

Beispiel-Nutzung:

```
<DSLInput
  onLoad={(text) => {
    const result = parseDSL(text);
    setNFA(result.nfa);
  }}
  onParseResult={(result) => {
```

```
    if (!result.success) {
      showError(result.error);
    }
  }}
/>
```

---

5. TypeScript-Typen

5.1 Kern-Typen

```
// types.ts

interface NFA {
  name?: string;
  regex?: string;
  states: string[];
  startState: string;
  acceptStates: string[];
  alphabet: string[];
  transitions: Transition[];
  hasEpsilon: boolean;
}

interface DFA {
  name?: string;
  regex?: string;
  states: string[];      // Potenzmengen
  startState: string;
  acceptStates: string[];
  alphabet: string[];
  transitions: Transition[];
}

interface Transition {
  from: string;
  to: string;
  symbol: string;
}

type ParseResult =
  | { success: true; nfa: NFA }
  | { success: false; error: string };
```

5.2 Interne Parser-Typen

```
interface StateInfo {
  name: string;
  isStart: boolean;
  isAccept: boolean;
}

interface ParsedLine {
  states: StateInfo[];
  symbols: string[];
}
```

---

6. Testing

6.1 Test-Struktur

```
tests/
├── Unit Tests
│   ├── dslParser.test.ts          # Parser-Logik
│   ├── strict-validation.test.ts  # Validierungsregeln
│   └── consistency-validation.test.ts
│
└── Component Tests
    ├── DSLInput.test.tsx          # UI-Komponente
    └── GraphViewer.test.tsx       # Visualisierung

src/
└── Component Tests (co-located)
    ├── core/parser/dslParser.test.ts
    ├── components/FileInput/DSLInput.test.tsx
    └── components/GraphViewer/GraphViewer.test.tsx
```

6.2 Tests ausführen

```
# Alle Tests
npm test

# Mit Watch-Mode
npm test -- --watch

# Spezifische Datei
npm test -- dslParser.test.ts

# Coverage-Report
npm run test:coverage

# UI für Tests
npm run test:ui
```

6.3 Test-Beispiel

```
import { describe, it, expect } from 'vitest';
import { parseDSL } from './dslParser';

describe('dslParser', () => {
  it('should parse simple NFA', () => {
    const input = `.q0 -a> (q1);`;
    const result = parseDSL(input);

    expect(result.success).toBe(true);
    if (result.success) {
      expect(result.nfa.states).toEqual(['q0', 'q1']);
      expect(result.nfa.startState).toBe('q0');
      expect(result.nfa.acceptStates).toContain('q1');
    }
  });
});
```

6.4 Mocking

ReactFlow mocken:

```
vi.mock('reactflow', () => ({
  __esModule: true,
  default: (props: any) => (
    <div data-testid="reactflow">
      {props.children}
    </div>
  ),
  MiniMap: () => <div data-testid="minimap" />,
  Controls: () => <div data-testid="controls" />,
  // ...
}));
```

Dagre mocken:

```
const mockNodePositions = new Map();

vi.mock('dagre', () => ({
  __esModule: true,
  default: {
    graphlib: { Graph: MockGraphClass },
    layout: vi.fn(),
  },
}));
```

---

7. Build & Deployment

7.1 Development Build

```
# Dev-Server mit HMR
npm run dev

# Typ-Checking
npm run build  # tsc -b && vite build
```

7.2 Production Build

```
# Build für Production
npm run build

# Output: dist/
# → index.html
# → assets/*.js
# → assets/*.css
```

7.3 Build testen

```
# Preview des Production Builds
npm run preview
# → http://localhost:4173
```

7.4 GitLab CI/CD

.gitlab-ci.yml:

```
stages:
  - install
  - test
  - build

install_dependencies:
  stage: install
  script:
    - npm ci
  artifacts:
    paths:
      - node_modules/
    expire_in: 1 day

run_tests:
  stage: test
  dependencies:
    - install_dependencies
  script:
    - npm test -- --run
    - npm run test:coverage

build_project:
  stage: build
  dependencies:
    - install_dependencies
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
    expire_in: 1 week
```

---

8. Code-Konventionen

8.1 Naming Conventions

```
// Components: PascalCase
GraphViewer.tsx
DSLInput.tsx

// Functions: camelCase
parseDSL()
layoutWithDagre()

// Constants: UPPER_SNAKE_CASE
const MAX_FILE_SIZE = 1024 * 1024;

// Types/Interfaces: PascalCase
interface NFA { }
type ParseResult = ...
```

8.2 Datei-Organisation

```
// Imports gruppieren
```

```
import React from 'react';                // External
import { parseDSL } from '@/core';        // Internal
import type { NFA } from '@/types';       // Types
import './styles.css';                    // Styles

// Component
export default function MyComponent() {
  // Hooks
  const [state, setState] = useState();

  // Handler
  const handleClick = () => { };

  // Render
  return <div>...</div>;
}
```

8.3 TypeScript Best Practices

```
// ☑ Gut: Explizite Typen für Funktionen
function parseLine(line: string, num: number): ParsedLine {
  // ...
}

// ☑ Gut: Type Guards
function isNFA(obj: any): obj is NFA {
  return obj && typeof obj.startState === 'string';
}

// ✗ Vermeiden: any ohne Grund
const data: any = fetchData();  // Schlecht

// ☑ Besser: unknown mit Type Guard
const data: unknown = fetchData();
if (isNFA(data)) {
  // data ist jetzt NFA
}
```

8.4 React Best Practices

```
// ☑ Gut: Destructuring von Props
function MyComponent({ nfa, onLoad }: Props) { }

// ☑ Gut: Early Returns
if (!nfa) return <div>No NFA</div>;

// ☑ Gut: Memoization für teure Berechnungen
const positions = useMemo(
  () => layoutWithDagre(states, transitions),
  [states, transitions]
);

// ✗ Vermeiden: Inline Functions in Props
<button onClick={() => doSomething()}>  // Neu bei jedem Render

// ☑ Besser: useCallback
const handleClick = useCallback(() => {
  doSomething();
```

```
}, [dependencies]);
```

---

9. Debugging

9.1 Browser DevTools

```
// Console Logging
console.log('NFA:', nfa);
console.table(transitions);

// React DevTools
// Component-Hierarchie inspizieren
// Props und State überwachen
```

9.2 Vitest Debugging

```
# Debug einzelner Test
npm test -- --inspect-brk dslParser.test.ts

# UI für Tests
npm run test:ui
```

9.3 TypeScript Errors

```
# Type-Check ohne Build
npx tsc --noEmit

# Spezifische Datei
npx tsc --noEmit src/core/parser/dslParser.ts
```

---

10. Performance-Optimierung

10.1 React Performance

```
// Memoization für teure Berechnungen
const nodes = useMemo(
  () => createNodes(nfa, positions),
  [nfa, positions]
);

// Callback-Memoization
const handleNodeClick = useCallback((id: string) => {
  console.log('Clicked:', id);
}, []);

// Component-Memoization
const MemoizedGraph = React.memo(GraphViewer);
```

10.2 Graph Performance

```
// Lazy Loading für große Graphen
const visibleNodes = nodes.slice(0, 100);
```

```
// Debouncing für Layout-Updates
const debouncedLayout = debounce(layoutWithDagre, 300);
```

---

11. Erweiterungen

11.1 Neue Komponente hinzufügen

```
# Datei erstellen
touch src/components/MyComponent/MyComponent.tsx
touch src/components/MyComponent/MyComponent.test.tsx

# Test erstellen
npm test -- MyComponent.test.tsx
```

11.2 Neues Feature entwickeln

1. Branch erstellen: `git checkout -b feature/my-feature`
2. Implementierung in `src/`
3. Tests in `tests/` oder co-located
4. Dokumentation aktualisieren
5. Merge Request erstellen

11.3 Parser erweitern

```
// In dslParser.ts
function parseNewFeature(token: string): FeatureInfo {
  // Implementierung
}

// Tests hinzufügen
describe('New Feature', () => {
  it('should parse new syntax', () => {
    // Test
  });
});
```

---

12. Troubleshooting

12.1 Build-Fehler

```
# Cache löschen
rm -rf node_modules dist
npm install

# TypeScript-Fehler
npx tsc --noEmit
```

12.2 Test-Fehler

```
# Coverage neu generieren
rm -rf coverage
```

```
npm run test:coverage

# Mock-Probleme
# → Überprüfen Sie vi.mock() Aufrufe
```

12.3 Hot Module Reload (HMR) funktioniert nicht

```
# Vite-Config überprüfen
# Server neu starten
npm run dev -- --force
```

---

13. API-Referenz

13.1 dslParser

```
/**
 * Parsed AEF-Format zu NFA
 * @param input - AEF-String
 * @returns ParseResult mit NFA oder Fehler
 */
export function parseDSL(input: string): ParseResult
```

13.2 layoutWithDagre

```
/**
 * Berechnet automatisches Layout für Graph
 * @param states - Liste der Zustände
 * @param transitions - Liste der Transitionen
 * @returns Map von Zustand zu Position
 */
function layoutWithDagre(
  states: string[],
  transitions: Transition[]
): Map<string, { x: number; y: number }>
```

---

14. Ressourcen

14.1 Externe Dokumentation

- React 19 Docs
- TypeScript Handbook
- Vitest Guide
- ReactFlow Documentation
- Dagre Wiki

Version: 1.0
Letzte Aktualisierung: Januar 2026